

Nested Transactions in SQL Server with Examples

In this article, I am going to discuss **Nested Transactions in SQL Server** with Examples. Please read our previous article where we discussed the Different [Types of Transaction Modes in SQL Server](#) with examples. As part of this article, we are going to discuss two important concepts i.e. **nested transaction** in SQL Server and **how to perform a Partial Rollback** in SQL Server.

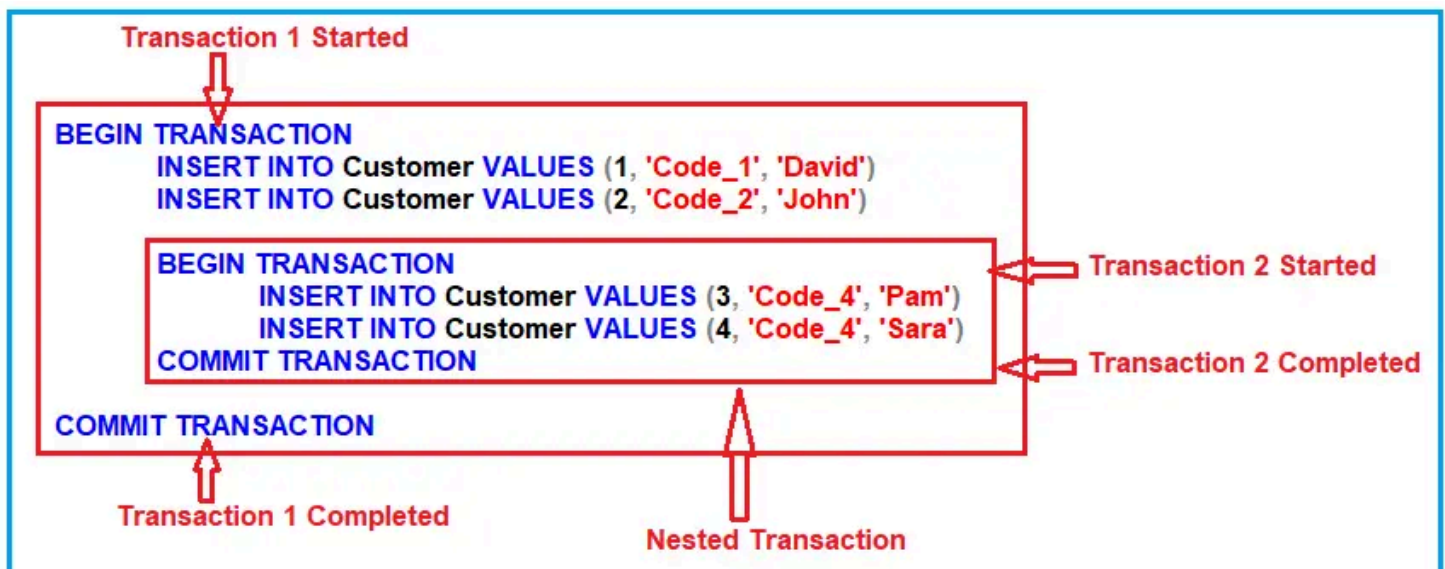
Nested Transaction in SQL Server:

It is also possible to put one transaction within another transaction in SQL Server and when we do so, it is called a nested transaction. We are going to use the following Customer table to understand the nested transaction in SQL Server. Please use the below SQL Script to create the Customer table.

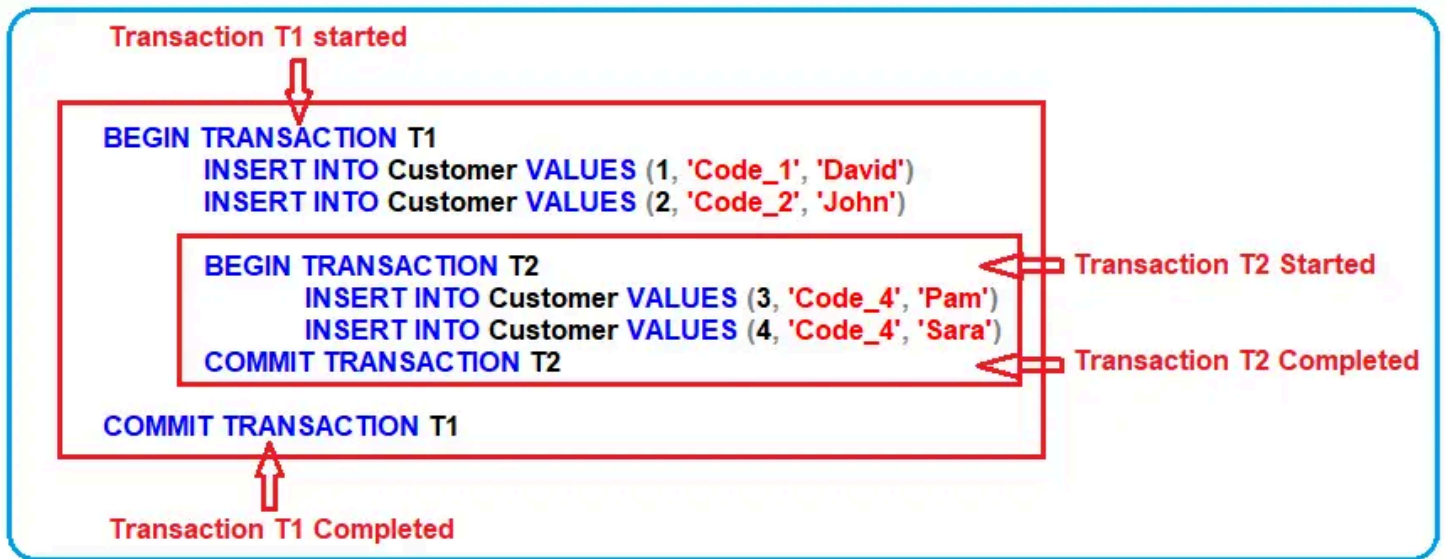
```
CREATE TABLE Customer
(
    CustomerID INT PRIMARY KEY,
    CustomerCode VARCHAR(10),
    CustomerName VARCHAR(50)
)
```

Understanding Nested Transaction in SQL Server:

In order to understand the nested transaction, please have a look at the following image where we have one transaction within another transaction. As you can see in the below image, first transaction 1 started and it inserts two records into the Customer table. Then transaction 2 started and it also inserts two records into the table. Then transaction 2 is committed first and then transaction 3 is committed.



When you have a lot of transactions in your query then definitely it will become more confusing. This is because you have many Begin Transaction and Commit Transaction statements in your query. So, what you can do here is, you can provide an explicit name to your transactions as shown in the below image.



So, when you have a big SQL Query with many transactions, and if you provide a name to your transactions as shown in the above image, then you can easily identify which commit transaction statement is for which begin transaction statement and moreover, your code is going to be more readable.

Points to remember:

When the inner transactions do a commit, it does not physically commit the transaction. That means we can say that nothing happens when the inner transactions do a commit. When the outer transaction does a commit, it actually commits a physical transaction and any data modified is saved permanently into the database. So, only the outer commit transaction commits the transaction.

What happens when the inner transactions commit in SQL Server?

There is something called transaction count whose value will decrease when the inner transaction commits. We have a global variable called **@@TRANCOUNT** which holds the value of the number of open transactions.

```
BEGIN TRANSACTION T1
  INSERT INTO Customer VALUES (10, 'Code_10', 'Ramesh')
  INSERT INTO Customer VALUES (11, 'Code_11', 'Suresh')

  BEGIN TRANSACTION T2
    INSERT INTO Customer VALUES (12, 'Code_12', 'Priyanka')
    INSERT INTO Customer VALUES (13, 'Code_13', 'Preety')
    PRINT @@TRANCOUNT -- Here TRANCOUNT value 2

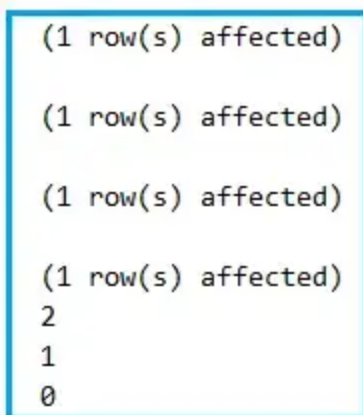
  COMMIT TRANSACTION T2 -- This does not physically commit
  PRINT @@TRANCOUNT -- Here TRANCOUNT value 1

COMMIT TRANSACTION T1 -- This does a physically commit
PRINT @@TRANCOUNT -- Here TRANCOUNT value 0
```

Let us execute the above code and see the output.

```
BEGIN TRANSACTION T1
    INSERT INTO Customer VALUES (10, 'Code_10', 'Ramesh')
    INSERT INTO Customer VALUES (11, 'Code_11', 'Suresh')
    BEGIN TRANSACTION T2
        INSERT INTO Customer VALUES (12, 'Code_12', 'Priyanka')
        INSERT INTO Customer VALUES (13, 'Code_13', 'Preety')
        PRINT @@TRANCOUNT -- Here TRANCOUNT value 2
    COMMIT TRANSACTION T2 -- This does not physically commit
    PRINT @@TRANCOUNT -- Here TRANCOUNT value 1
COMMIT TRANSACTION T1 -- This does a physically commit
PRINT @@TRANCOUNT -- Here TRANCOUNT value 0
```

Output: As you can see, first it will print the transaction count value as 2 as we have two open transactions initially and once the inner transaction commits, the value is reduced to 1 and once the outer transaction commits the value is reduced to 0.



```
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)
(1 row(s) affected)
2
1
0
```

Note: The inner commit does not do anything and that makes sense. Think that if the inner transaction commits the data physically into the database and later if the outer transaction rollback then how difficult it to roll back the inner transaction data.

What is the use of nested transactions in SQL Server?

The nested transactions are basically used with checkpoints or save points for a partial rollback in SQL Server.

What are SavePoints in SQL Server?

The **SAVE TRANSACTION** in SQL Server is used for dividing (or) breaking a transaction into multiple units so that the user has a chance of roll backing the transaction up to a specified point. That means using SavePoints Transaction we can roll back a part of a transaction instead of the entire transaction.

In order to create a checkpoint or save point or you can say logical point, you need to use the **SAVE TRANSACTION** command followed by the save point name. In order to understand this better please have a look at the following image. As you can see in the following image, first we start a transaction and then create three Save Points (Logical Points and Check Point) using the Save Transaction command.

BEGIN TRANSACTION

SAVE TRANSACTION SavePoint1

INSERT INTO Customer VALUES (1, 'Code_1', 'Ramesh')

INSERT INTO Customer VALUES (2, 'Code_2', 'Suresh')

SAVE TRANSACTION SavePoint2

INSERT INTO Customer VALUES (3, 'Code_3', 'Priyanka')

INSERT INTO Customer VALUES (4, 'Code_4', 'Preety')

SAVE TRANSACTION SavePoint3

INSERT INTO Customer VALUES (5, 'Code_5', 'John')

INSERT INTO Customer VALUES (6, 'Code_6', 'David')

Case1 ROLLBACK TRANSACTION SavePoint1

Case2 ROLLBACK TRANSACTION SavePoint2

Case3 ROLLBACK TRANSACTION SavePoint3

As per the definition, it will roll back the statements which are present after the savepoint and before the Rollback Statement. As we created three logical points (SavePoint1, SavePoint2, and SavePoint3), let us understand what happens when we execute a particular savepoint with the rollback command.

CASE1:

ROLLBACK TRANSACTION SavePoint1

When we execute the above Rollback command, then it will roll back the statements which are starting from SavePoint1, and before the rollback statement i.e. in our example, it will roll back all the 6 Insert statements.

CASE2:

ROLLBACK TRANSACTION SavePoint2

When we execute the above Rollback transaction statement, then the transaction will roll back the statements which are starting from SavePoint2 and end before the Rollback statement. That means, in this case, it will roll back 4 Insert statements.

CASE3:

ROLLBACK TRANSACTION SavePoint3

When we execute the above Rollback statement, it will roll back the statements which are present after the SavePoint3 and before the Rollback Command. That means the transaction will roll back two insert statements.

Partial Rollback Example in SQL Server:

First, delete all the data from the Customer table by executing the following SQL Query.

DELETE FROM Customer;

Then execute the following statements. The following statement will create a transaction with three save points.

```
BEGIN TRANSACTION
SAVE TRANSACTION SavePoint1
    INSERT INTO Customer VALUES (1, 'Code_1', 'Ramesh')
    INSERT INTO Customer VALUES (2, 'Code_2', 'Suresh')
SAVE TRANSACTION SavePoint2
    INSERT INTO Customer VALUES (3, 'Code_3', 'Priyanka')
    INSERT INTO Customer VALUES (4, 'Code_4', 'Preety')
SAVE TRANSACTION SavePoint3
    INSERT INTO Customer VALUES (5, 'Code_5', 'John')
    INSERT INTO Customer VALUES (6, 'Code_6', 'David')
```

Now, execute the following rollback statement which should roll back four records.

Rollback Transaction SavePoint2;

Once you execute the above rollback statement, then commit the transaction by executing the below commit transaction statement which will commit two records into the database.

Commit Transaction;

Now check the Customer table and there should be two records as shown below.

CustomerID	CustomerCode	CustomerName
1	Code_1	Ramesh
2	Code_2	Suresh

Note: The savepoint name in SQL Server can have a maximum of 32 characters. If a savepoint variable is used instead of a savepoint name and a string with more than 32 characters is assigned to it, only the first 32 characters will define the savepoint name and the remaining characters will be ignored.

SavePoint with Nested Transaction in SQL Server:

It is also possible to use SavePoint with nested transactions in SQL Server. Let us understand this with an example. First, as always delete the data from the Customer table by executing the below SQL statement.

DELETE FROM Customer;

Create nested transactions in SQL Server using the save point by executing the below SQL query.

```
BEGIN TRANSACTION T1
    SAVE TRANSACTION SavePoint1
        INSERT INTO Customer VALUES (10, 'Code_10', 'Ramesh')
        INSERT INTO Customer VALUES (11, 'Code_11', 'Suresh')

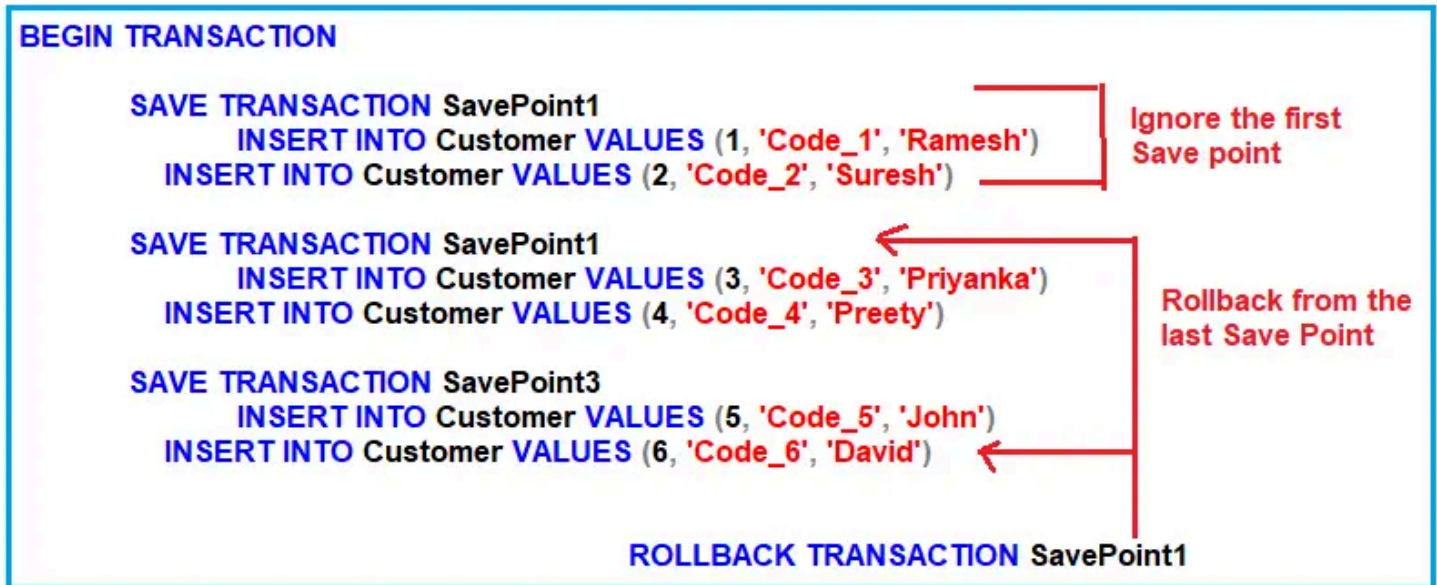
    BEGIN TRANSACTION T2
        SAVE TRANSACTION SavePoint2
            INSERT INTO Customer VALUES (12, 'Code_12', 'Priyanka')
            INSERT INTO Customer VALUES (13, 'Code_13', 'Preety')
    COMMIT TRANSACTION T2
    ROLLBACK TRANSACTION SavePoint2
```

COMMIT TRANSACTION T1

When we execute the above statement, it should roll back the 3rd and 4th records while the 1st and 2nd records should be committed into the database.

SQL Server transaction savepoints with the same savepoint name

It is also possible in the SQL server to have multiple savepoints with the same name, but in the case of a rollback, the transaction will be rolled back to the latest SAVE TRANSACTION using that name. In order to understand this better please have a look at the following image.



Here, we created two savepoints with the same name i.e. SavePoint1. When we roll back SavePoint1 then it will roll back the data from the 2nd save point.

Example: Savepoints with the same savepoint name in SQL Server Transaction

First, delete all the data from the Customer table

DELETE FROM Customer;

Then execute the following statements.

```
BEGIN TRANSACTION
SAVE TRANSACTION SavePoint1
    INSERT INTO Customer VALUES (1, 'Code_1', 'Ramesh')
    INSERT INTO Customer VALUES (2, 'Code_2', 'Suresh')
SAVE TRANSACTION SavePoint1
    INSERT INTO Customer VALUES (3, 'Code_3', 'Priyanka')
    INSERT INTO Customer VALUES (4, 'Code_4', 'Preety')
SAVE TRANSACTION SavePoint3
    INSERT INTO Customer VALUES (5, 'Code_5', 'John')
    INSERT INTO Customer VALUES (6, 'Code_6', 'David')
ROLLBACK TRANSACTION SavePoint1
COMMIT TRANSACTION
```

Note: The SAVEPOINT command in SQL Server is used to temporarily save a transaction so that the user can roll back to that point if required.

In the next article, I am going to discuss [ACID Properties](#) in detail. Here, in this article, I try to explain the **Nested Transactions in SQL Server** with Examples. I hope this Nested Transactions in SQL Server with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.